Pyroclast: GPU-Accelerated Monte Carlo Simulation for Lava-Flow Habitat Impact Assessment

Emanuele Galiano

April 27, 2026

Contents

1 Monte Carlo Kernel **2**
1.1 Problem Statement . . . 2
1.2 1-D Kernel . . . 2
1.2.1 Launch Scheme . . . 2
1.2.2 Algorithm for Work-Item r . . . 2
1.2.3 Source . . . 3
1.2.4 `p_vec` Reads and Cache Behaviour . . . 3
1.3 2-D Kernel Attempt . . . 3
1.3.1 Motivation . . . 3
1.3.2 Need for an Intermediate Buffer . . . 3
1.3.3 Source — Kernel 1: `mc_partial_sums` . . . 4
1.3.4 Intermediate Buffer Size . . . 5
1.3.5 Why Barriers Do Not Solve the Problem . . . 5
1.3.6 Comparison with the 1-D Kernel . . . 5
1.3.7 Experimental Results . . . 6

A Library Architecture **7**

Chapter 1

Monte Carlo Kernel

1.1 Problem Statement

After the preprocessing pipeline applies stream compaction to the invasion probability raster, each habitat is represented as a dense vector $\mathbf{p} \in [0,1]^{N_c}$, where p_k is the probability that active cell k is reached by the lava flow. The Monte Carlo kernel estimates the probability that the *invaded fraction* of cells strictly exceeds a critical threshold θ:

$$\Pr\left(\frac{1}{N_c}\sum_{k=0}^{N_c-1} X_k > \theta\right), \qquad X_k \sim \text{Bernoulli}(p_k). \tag{1.1}$$

The estimate is obtained by running R independent trials and counting how many exceed the threshold.

1.2 1-D Kernel

1.2.1 Launch Scheme

The NDRange is one-dimensional: each work-item handles one complete trial.

Global NDRange	$\lceil R/\texttt{WG_SIZE}\rceil \times \texttt{WG_SIZE}$
Local NDRange	$\texttt{WG_SIZE} = 256$

1.2.2 Algorithm for Work-Item r

1. Seed MWC64X at stream position `base_offset` $+ r \cdot N_c$ with length N_c: each work-item occupies a non-overlapping segment of the RNG sequence.
2. For each cell $k = 0, \ldots, N_c - 1$: draw $x \sim U(0,1)$ and compute $b_k = \mathbf{1}[x \le p_k]$.
3. Count invaded cells: `invaded` $= \sum_k b_k$.
4. Compute the success bit: $s_r = \mathbf{1}[\texttt{invaded}/N_c > \theta]$.
5. Participate in the work-group tree reduction; thread 0 issues `atomic_add` on the global counter.

1.2.3 Source

Listing 1.1: 1-D kernel — `monte_carlo.cl`

```
__kernel __attribute__((reqd_work_group_size(WG_SIZE, 1, 1)))
void monte_carlo_run(
    __global const float* p_vec,
    __global int*         count,
    const uint            n_cells,
    const float           threshold,
    const ulong           base_offset,
    const uint            n_runs)
{
    uint r   = get_global_id(0);
    uint lid = get_local_id(0);

    int my_result = 0;
    if (r < n_runs)
        my_result = _run_trial(p_vec, n_cells, threshold,
                               base_offset + (ulong)r * n_cells);

    __local int scratch[WG_SIZE];
    scratch[lid] = my_result;
    barrier(CLK_LOCAL_MEM_FENCE);
    _tree_reduce(scratch, lid);

    if (lid == 0)
        atomic_add(count, scratch[0]);
}
```

1.2.4 `p_vec` Reads and Cache Behaviour

At each iteration k, all 256 work-items in the work-group read the *same* element `p_vec[k]`: the value is loaded into L1 cache once and broadcast to all threads at no additional cost.

1.3 2-D Kernel Attempt

1.3.1 Motivation

In the 1-D kernel each work-item samples N_c cells sequentially. The idea behind the 2-D kernel is to parallelise the cell axis as well: each work-item samples exactly *one* pair (run r, cell k), exploiting a two-dimensional NDRange.

Global NDRange	$\lceil R/\texttt{WG_R}\rceil \times \texttt{WG_R}\,, \quad \lceil N_c/\texttt{WG_C}\rceil \times \texttt{WG_C}$
Local NDRange	$(\texttt{WG_R},\ \texttt{WG_C}) = (8,\ 32)$

1.3.2 Need for an Intermediate Buffer

When $N_c > \texttt{WG_C}$, the cells of a single run are spread across multiple work-groups along the k axis. For example, with $N_c = 96$ and $\texttt{WG_C} = 32$:

Work-group	Runs	Cells
WG(0, 0)	0 . . . 7	0 . . . 31
WG(0, 1)	0 . . . 7	32 . . . 63
WG(0, 2)	0 . . . 7	64 . . . 95

To determine whether run 0 exceeded the threshold, the partial counts from all three work-groups must be summed. In OpenCL 1.2 **work-groups cannot synchronise with each other** within a single kernel launch: `barrier` is local to a work-group and has no global visibility. The only available global synchronisation point is the *end of a kernel launch*. The solution is therefore to write partial counts to a global buffer `g_partial` and launch a second kernel that reduces them.

1.3.3 Source — Kernel 1: `mc_partial_sums`

Listing 1.2: 2-D kernel, first launch — `monte_carlo_2d.cl`

```
__kernel __attribute__((reqd_work_group_size(WG_R, WG_C, 1)))
void mc_partial_sums(
    __global const float* p_vec,
    __global int*         g_partial,   /* [n_groups_c][
       n_runs_padded] */
    const uint            n_cells,
    const uint            n_runs,
    const ulong           base_offset,
    const uint            n_runs_padded)
{
    uint r     = get_global_id(0);
    uint k     = get_global_id(1);
    uint lid_r = get_local_id(0);
    uint lid_c = get_local_id(1);

    mwc64x_state_t rng;
    MWC64X_SeedStreams(&rng,
        base_offset + (ulong)r * n_cells + k, 1UL);

    int my_bit = 0;
    if (r < n_runs && k < n_cells)
        my_bit = (_to_float(MWC64X_NextUint(&rng)) <= p_vec[k])
                 ? 1 : 0;

    __local int scratch[WG_R * WG_C];
    scratch[lid_r * WG_C + lid_c] = my_bit;
    barrier(CLK_LOCAL_MEM_FENCE);
    _reduce_cols(scratch, lid_r, lid_c);

    if (lid_c == 0 && r < n_runs)
        g_partial[get_group_id(1) * n_runs_padded + r]
            = scratch[lid_r * WG_C];
}
```

1.3.4 Intermediate Buffer Size

The `g_partial` buffer uses a transposed layout $[n_\text{groups_c}][n_\text{runs_padded}]$, where

$$n_\text{groups_c} = \left\lceil \frac{N_c}{\texttt{WG_C}} \right\rceil, \qquad n_\text{runs_padded} = \left\lceil \frac{R}{\texttt{WG_R}} \right\rceil \times \texttt{WG_R}. \tag{1.2}$$

With real project data (where N_c may have $\approx 200\,000$ active cells after stream compaction, batch size $R = 100\,000$, $\texttt{WG_C} = 32$, $\texttt{WG_R} = 8$):

$$\begin{aligned} n_\text{groups_c} &= \lceil 200\,000/32 \rceil = 6\,250 \\ n_\text{runs_padded} &= \lceil 100\,000/8 \rceil \times 8 = 100\,000 \\ |\texttt{g_partial}| &= 6\,250 \times 100\,000 \times 4\,\text{B} = \mathbf{2.5}\,\text{GB}. \end{aligned}$$

This buffer is allocated on the CPU via `np.zeros`, copied to the GPU, and released on *every batch*. With 10 batches per habitat the cumulative transfer exceeds 25 GB, making the 2-D kernel orders of magnitude slower than the 1-D kernel.

1.3.5 Why Barriers Do Not Solve the Problem

The primitive `barrier(CLK_LOCAL_MEM_FENCE)` synchronises only the work-items *within the same work-group*. Work-groups `WG(0,0)`, `WG(0,1)`, and `WG(0,2)` from the example above execute independently: OpenCL 1.2 provides no global barrier primitive that could force them to wait for one another within a single kernel launch. The only global synchronisation available is the kernel launch boundary itself, which is why the two-kernel design is necessary.

1.3.6 Comparison with the 1-D Kernel

The 2-D kernel offers no architectural advantage over the 1-D kernel for two reasons:

1. **Cache broadcast.** In the 1-D kernel, at iteration k all 256 work-items read `p_vec[k]`: a single cache line is loaded once and the value is broadcast to all threads at no extra cost. The 2-D kernel reads 32 consecutive floats per cycle (coalesced), which is not meaningfully better in terms of effective throughput.

2. **Intermediate buffer cost.** Parallelising the cell axis introduces an $O(N_c \times R)$ intermediate buffer that eliminates any computational advantage on real-world data.

The 1-D kernel therefore remains the optimal choice for this problem.

1.3.7 Experimental Results

Test Environment

Component	Details
CPU	Intel Core i7-12700H (20 threads, 4.70 GHz)
GPU	NVIDIA GeForce RTX 4050 Max-Q / Mobile
RAM	15.24 GiB DDR5
OpenCL	1.2
Python	3.14.4
`MC_RUNS`	1 000
`MC_THRESHOLD`	0.009
`MC_BATCHES`	10
Habitats processed	4
Kernel launches	40 (4 habitats $\times$ 10 batches)

Timing and Bandwidth

Kernel	Launches	Mean (ms)	Min (ms)	Bandwidth (GB/s)
`monte_carlo_run` (1-D)	40	2.273	0.297	15.19
`mc_partial_sums` (2-D)	40	185.379	13.259	0.19
`mc_threshold_count` (2-D)	40	0.087	0.010	12.90

The 2-D pipeline is $\approx 82\times$ slower than the 1-D kernel (speedup $= 0.01\times$). The bottleneck is `mc_partial_sums`, whose low effective bandwidth (0.19 GB/s versus 15.19 GB/s for the 1-D kernel) reflects the allocation and transfer latency of the 2.5 GB `g_partial` buffer per batch, as discussed in Section 1.3.

Appendix A

Library Architecture

(Appendix placeholder — to be expanded.)
DUMMY CITATION [1]

Bibliography

[1] Harry Percival and Bob Gregory. *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices*. O'Reilly Media, Sebastopol, CA, 1 edition, 2020.